Практическая работа №3.

Создание отношений, установка связей с помощью запросов в СУБД mySQL.

Для успешной работы необходимо зайти на сайт https://beget.com/ru и нажать кнопку вход. Для входа воспользоваться следующими данными: логин**: damsel8s**, пароль: **YvF8QChu**. Далее выбрать меню MySql, зайти и создать базу по выбранному варианту.

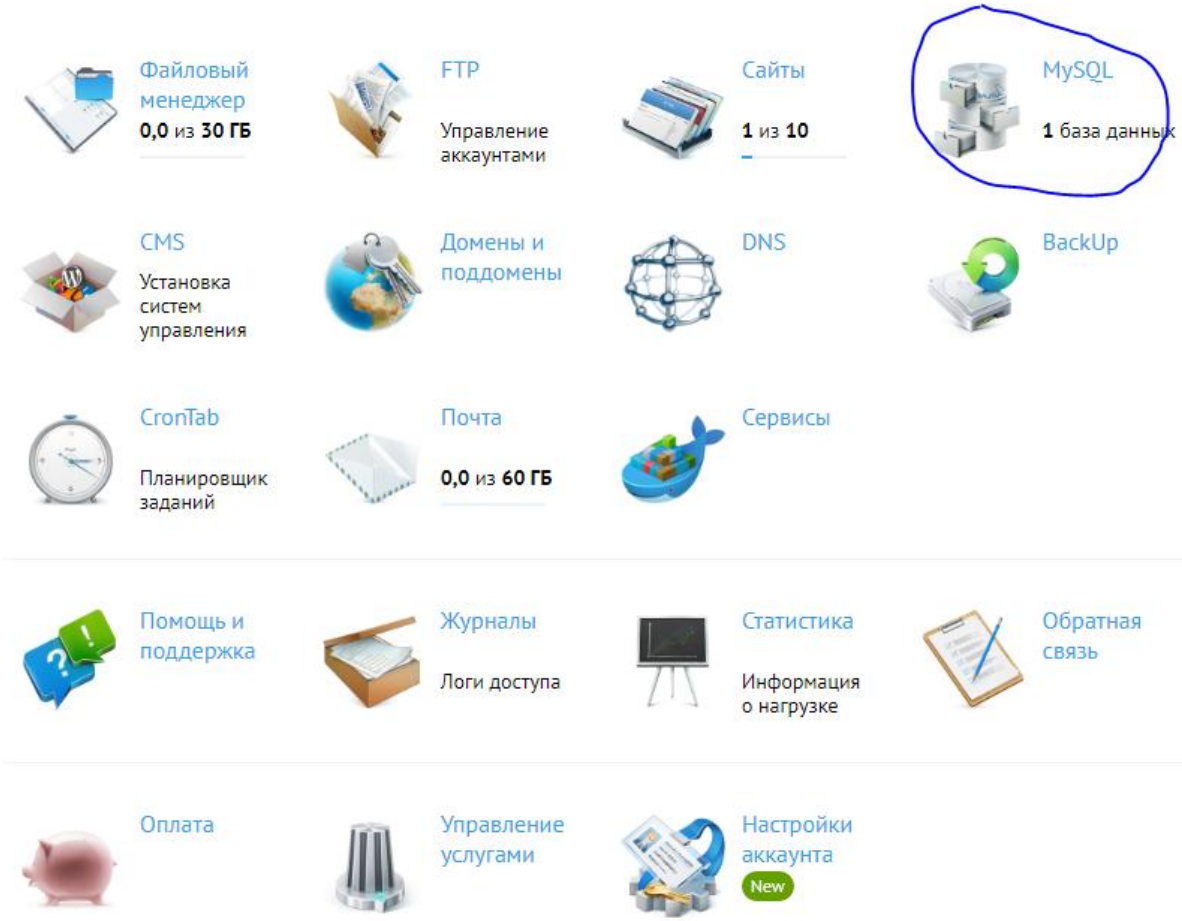


Рисунок 1 – Панель управления

Далее необходимо создать базу по своей фамилии, а также придумать пароль.

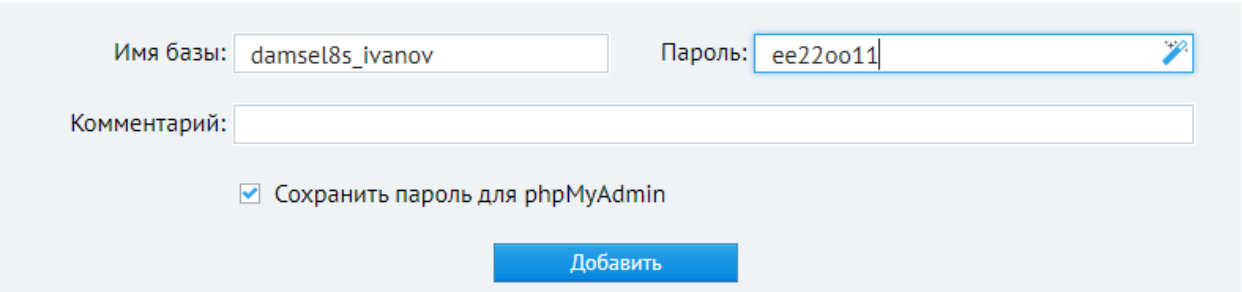


Рисунок 2 – Первичные настройки базы.

После этого необходимо нажать пункт меню «перейти в phpmyadmin».

Далее необходимо будет создать запросы на создание таблиц, запросы на добавление содержимого. В первой работе были разработаны инфологическая и даталогическая модели с помощью которых в дальнейшем возможно преобразование.

Создание отношений.

Для корректного создания таблиц стоит руководствоваться инструкциями запроса **CREATE TABLE NAME_TABLE (NAME_VALUE TYPE_VALUE, NAME_VALUE1, TYPE_VALUE1)**. После этого необходимо установить нужные связи.

В phpmyadmin нужно перейти во вкладку SQL и начать вводить запрос. Предположим, необходимо создать отношение «маршрутная сеть» для аэропорта. Для этого нужна следующие атрибуты: id рейса, id города, куда направляется самолет и id перевозчика.

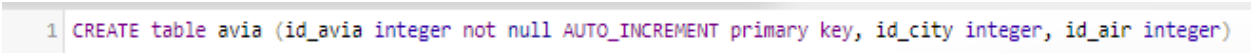


Рисунок 3 – Пример запроса на создание

Стоит отдельно отметить типы данных и назначения:

Символьные типы

- CHAR: представляет строку фиксированной длины.

Длина хранимой строки указывается в скобках, например, CHAR(10) - строка из десяти символов. И если в таблицу в данный столбец сохраняется строка из 6 символов (то есть меньше установленной длины в 10 символов), то строка дополняется 4 пробелами и в итоге все равно будет занимать 10 символов

Тип CHAR может хранить до 255 байт.

- VARCHAR: представляет строку переменной длины.

Длина хранимой строки также указывается в скобках, например, VARCHAR(10). Однако в отличие от CHAR хранимая строка будет занимать именно столько места, сколько необходимо. Например, если определенная длина в 10 символов, но в столбец сохраняется строка в 6 символов, то хранимая строка так и будет занимать 6 символов плюс дополнительный байт, который хранит длину строки.

Всего тип VARCHAR может хранить до 65535 байт.

Ряд дополнительных типов данных представляют текст неопределенной длины:

- TINYTEXT: представляет текст длиной до 255 байт.
- TEXT: представляет текст длиной до 65 КБ.
- MEDIUMTEXT: представляет текст длиной до 16 МБ
- LONGTEXT: представляет текст длиной до 4 ГБ

Числовые типы

- TINYINT: представляет целые числа от -128 до 127, занимает 1 байт

- BOOL: фактически не представляет отдельный тип, а является лишь псевдонимом для типа TINYINT(1) и может хранить два значения 0 и 1. Однако данный тип может также в качестве значения принимать встроенные константы TRUE (представляет число 1) и FALSE (предоставляет число 0).

Также имеет псевдоним BOOLEAN.

- TINYINT UNSIGNED: представляет целые числа от 0 до 255, занимает 1 байт
- SMALLINT: представляет целые числа от -32768 до 32767, занимает 2 байта
- SMALLINT UNSIGNED: представляет целые числа от 0 до 65535, занимает 2 байта
- MEDIUMINT: представляет целые числа от -8388608 до 8388607, занимает 3 байта
- MEDIUMINT UNSIGNED: представляет целые числа от 0 до 16777215, занимает 3 байта
- INT: представляет целые числа от -2147483648 до 2147483647, занимает 4 байта
- INT UNSIGNED: представляет целые числа от 0 до 4294967295, занимает 4 байта
- BIGINT: представляет целые числа от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807, занимает 8 байт
- BIGINT UNSIGNED: представляет целые числа от 0 до 18 446 744 073 709 551 615, занимает 8 байт
- DECIMAL: хранит числа с фиксированной точностью. Данный тип может принимать два параметра precision и scale: DECIMAL(precision, scale).

Параметр precision представляет максимальное количество цифр, которые может хранить число. Это значение должно находиться в диапазоне от 1 до 65.

Параметр scale представляет максимальное количество цифр, которые может содержать число после запятой. Это значение должно находиться в диапазоне от 0 до значения параметра precision. По умолчанию оно равно 0.

Данный тип также имеет псевдонимы NUMERIC, DEC, FIXED.

- FLOAT: хранит дробные числа с плавающей точкой одинарной точности от -3.4028 * 1038 до 3.4028 * 1038, занимает 4 байта

Может принимать форму FLOAT(M,D), где M - общее количество цифр, а D - количество цифр после запятой

.DOUBLE: хранит дробные числа с плавающей точкой двойной точности от -1.7976 * 10308 до 1.7976 * 10308, занимает 8 байт. Также может принимать форму DOUBLE(M,D), где M - общее количество цифр, а D - количество цифр после запятой.

Данный тип также имеет псевдонимы REAL и DOUBLE PRECISION, которые можно использовать вместо DOUBLE.

Типы для работы с датой и временем

- DATE: хранит даты с 1 января 1000 года до 31 деабря 9999 года (с "1000-01-01" до "9999-12-31"). По умолчанию для хранения используется формат yyyy-mm-dd. Занимает 3 байта.
- TIME: хранит время от -838:59:59 до 838:59:59. По умолчанию для хранения времени применяется формат "hh:mm:ss". Занимает 3 байта.
- DATETIME: объединяет время и дату, диапазон дат и времени - с 1 января 1000 года по 31 декабря 9999 года (с "1000-01-01 00:00:00" до "9999-12-31 23:59:59"). Для хранения по умолчанию используется формат "yyyy-mm-dd hh:mm:ss". Занимает 8 байт
- TIMESTAMP: также хранит дату и время, но в другом диапазоне: от "1970-01-01 00:00:01" UTC до "2038-01-19 03:14:07" UTC. Занимает 4 байта
- YEAR: хранит год в виде 4 цифр. Диапазон доступных значений от 1901 до 2155. Занимает 1 байт.

Тип Date может принимать даты в различных форматах, однако непосредственно для хранения в самой бд даты приводятся к формату "yyyy-mm-dd". Некоторые из принимаемых форматов:

- yyyy-mm-dd - 2018-05-25
- yyyy-m-dd - 2018-5-25
- yy-m-dd - 18-05-25

Составные типы

- ENUM: хранит одно значение из списка допустимых значений. Занимает 1-2 байта
- SET: может хранить несколько значений (до 64 значений) из некоторого списка допустимых значений. Занимает 1-8 байт.

Также стоит отметить особенности атрибутов:

- NOT NULL – не допускает пустых значений
- NULL – допускает возможность наличия пустых значений
- AUTO_INCREMENT – позволяет выставить автоматический инкремент для формировании значения атрибута

- PRIMARY KEY – назначает первичный ключ в таблице
- UNIQUE – назначает уникальные значения для атрибута
- INDEX – назначает атрибуту индекс, в частности вторичный ключ.

Создать подчиненные таблицы можно с помощью запроса с указанием связей с главной таблицей через внешний ключ FOREIGN KEY, связь между таблицами обозначается с помощью оператора REFERENCES.

```
CREATE table new (id_name integer not null AUTO_INCREMENT PRIMARY KEY, id_avia integer, name char(50),
foreign key (id_avia) REFERENCES avia (id_avia))
```

Рисунок 5 – Создать таблицу в режиме кода с указанием связи

Таким образом, алгоритм построения подчиненной таблицы:

1. указать имя и столбцы;
2. поставить запятую и указать в скобках наименование вторичного (внешнего) ключа;
3. после оператора REFERENCES указать таблицу, с которой создается связь и по какому ключевому полю в ней.

Для проверки можно перейти во вкладку «Дизайнер» и увидеть, что связь была успешно создана вместе с новой таблицей. Однако в текущем коде не было указано, что необходимо обеспечить каскадное добавление и удаление данных. Для того, чтобы это было доступным можно либо в интерфейсе связей настроить эту опцию, либо прописать в коде создания таблицы.

```
CREATE table new (id_name integer not null AUTO_INCREMENT PRIMARY KEY, id_avia integer, name char(50),
foreign key (id_avia) REFERENCES avia (id_avia) ON DELETE CASCADE ON UPDATE CASCADE)
```

Рисунок 6 – Создать таблицу в режиме кода с указанием связи и удаления с обновлением каскадом

В интерфейсе настройка будет выглядеть следующим образом:

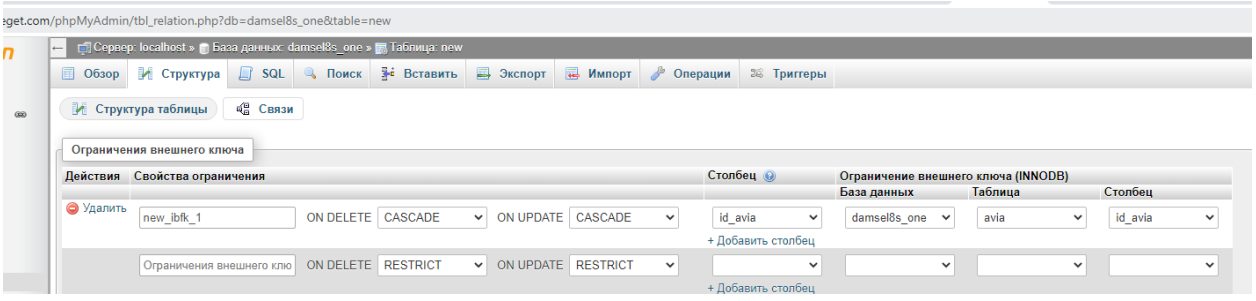


Рисунок 7 – Редактирование связей в режиме редактора

Во вкладке «дизайнер» необходимо настроить соответствующую связь. Связь должна быть организована между таблицами air и avia, а также должны быть обеспечены каскадные удаление и обновление. При это стоит помнить о том, что одна из этих таблиц является главной (родителем), а другая является потомком. В данном случае таблица air

будет родительской, а таблица avia является потомком. В этой связи в качестве внешнего ключа будет выступать атрибут id_air, который является первичным ключом и переходит в в отношение-потомок и там используется для моделирования связи. В данном случае будет действовать связь один-ко многим (один перевозчик может иметь множество рейсов). При этом указаны каскадные удаление и обновление. Каскадное обновление значит, что разрешается изменять значение первичного ключа в кортеже родительского отношения, если в отношении-потомке есть ссылающиеся на него кортежи. При этом будут изменены «по каскаду» значения внешних ключей во всех кортежах отношений-потомков, ссылающихся на обновляемый кортеж. Каскадное добавление значит, что разрешается удаление кортежа родительского отношения, если в отношении-потомке есть ссылающиеся на него кортежи. При этом будут удалены «по каскаду» все кортежи отношений-потомков, ссылающихся на удаляемый кортеж. В сложной базе данных структура будет представлена в виде дерева. То есть представляет собой сбалансированное дерево поиска, в котором каждый узел содержит множество ключей потомка. В конечном итоге база будет выглядеть именно в таком представлении. Связи между базами были отражены в даталогической модели и теперь переносятся на реальную модель.

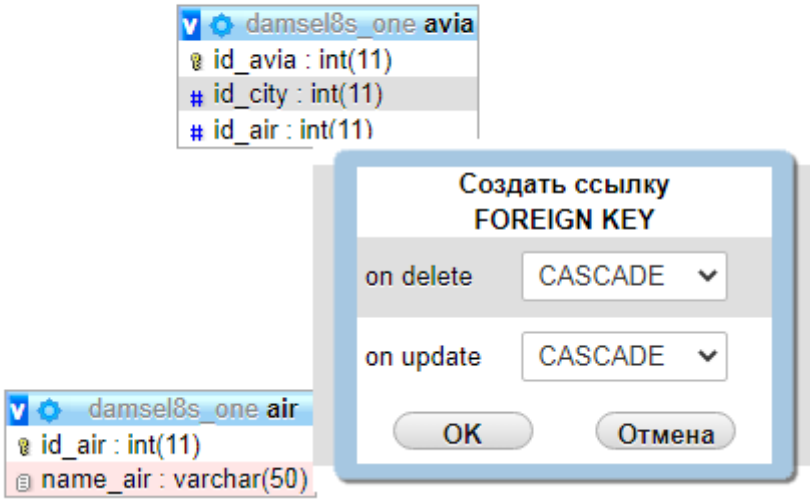


Рисунок 8 – Пример настройки схемы данных.

Задание.

Создать все необходимые таблицы в соответствии с типами данных, ограничениями и связями по ранее созданным моделям.